Experiment No : 04

Aim :

Learn use of maplotlib library as visualisation tool in data science pipeline to create contour plots, histograms and box plots.

Performance :

Density and Contour plots

Sometimes it is useful to display three-dimensional data in two dimensions using contours or color-coded regions.

There are three Matplotlib functions that can be helpful for this task: plt.contour for contour plots, plt.contourf for filled contour plots, and plt.imshow for showing images.

We will see examples of this type of plot in section below

```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np

plt.style.use('seaborn-v0_8-white')
```

```
# A 3D function to plot

def f(x, y):
    return np.sin(x) ** 10 + np.cos(10 + y * x) * np.cos(x)
```

A contour plot can be created with the plt.contour function. It takes three arguments: a grid of x values, a grid of y values, and a grid of z values.

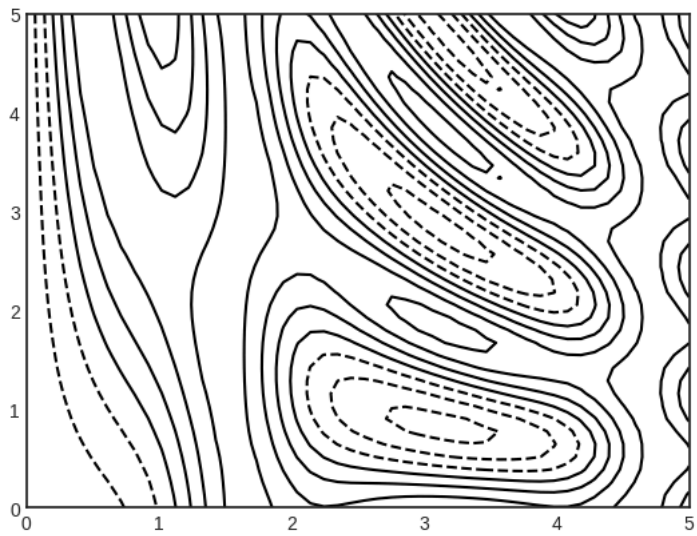
The x and y values represent positions on the plot, and the z values will be represented by the contour levels.

Perhaps the most straightforward way to prepare such data is to use the np.meshgrid function, which builds two-dimensional grids from one-dimensional arrays:

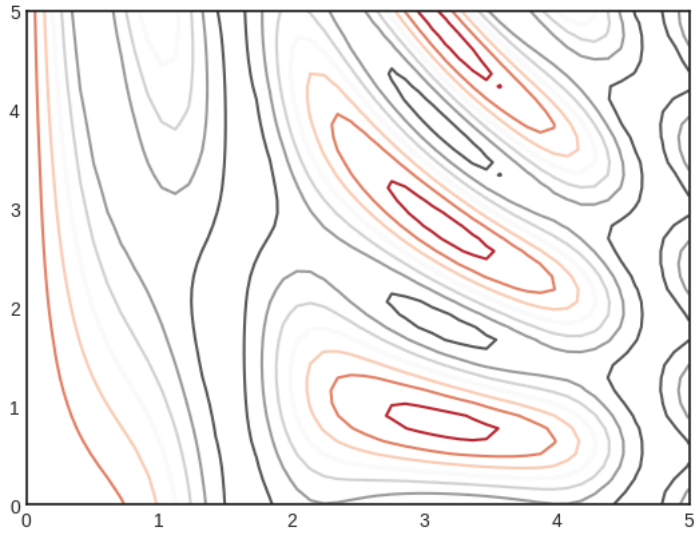
```
# x is set of 50 values between 0 and 5
# x is set of 40 values between 0 and 5

x = np.linspace(0, 5, 50)
y = np.linspace(0, 5, 40)

X, Y = np.meshgrid(x, y)
Z = f(X, Y)
```

Now let's look at this with a standard line-only contour plot:

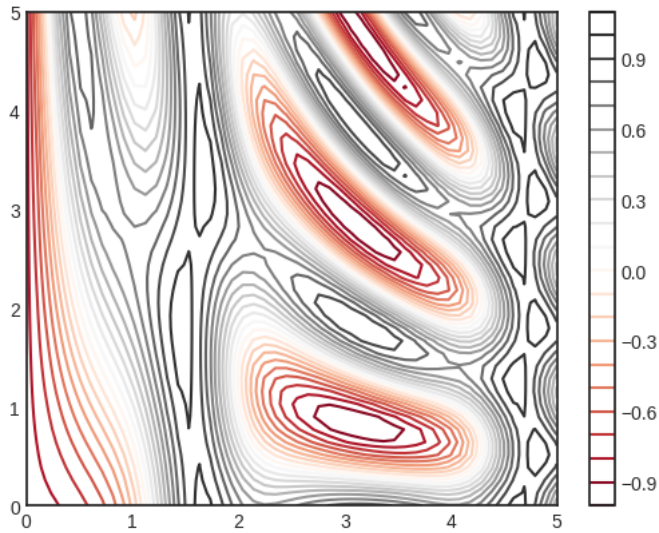
```
plt.contour(X, Y, Z, colors='black');
```



```
plt.contour(X, Y, Z, cmap='RdGy');
```




```
plt.contour(X, Y, Z, 20, cmap='RdGy');
plt.colorbar();
```



Answer Following Questions :

1.Explain what are the changes you observe in contour plots above 3 code cells ?

First Plot (plt.contour(X, Y, Z, colors='black')):

This is the most basic contour plot. All the contour lines are rendered in a

1.single color: black. This gives a stark, monochromatic view of the function's contours. The number of contour levels (i.e., the number of lines drawn) is set to the Matplotlib default, which typically results in 8 levels.

2.Second Plot (plt.contour(X, Y, Z, cmap='RdGy')):

In this plot, the colors='black' argument is replaced with cmap='RdGy'. This change introduces a colormap (RdGy stands for Red-Gray) to the contour lines. Instead of all lines being black, they are now colored based on their Z value. This provides more visual information about the magnitude of the function at different contours, with a gradient from red to gray. The number of contour levels remains at the default (8), similar to the first plot.

3.Third Plot (plt.contour(X, Y, Z, 20, cmap='RdGy'); plt.colorbar()):

This plot further refines the visualization by introducing two key changes: The argument 20 is added, which explicitly specifies that 20 contour levels should be drawn. This significantly increases the detail of the plot, showing more subtle changes in the function's surface compared to the default 8 levels. plt.colorbar() is called, which adds a color bar to the side of the plot. This color bar is crucial for interpreting the colormap (RdGy) and understanding which Z values correspond to which colors on the contour lines. It provides a visual legend for the contour levels.

2.What is size of X, Y and Z respectively ?

Based on the kernel state, the sizes of X, Y, and Z are:

X: (40, 50) Y: (40, 50) Z: (40, 50)

3.How does colorbar() method help in contour plot.

he colorbar() method is incredibly helpful in a contour plot, especially when you use a colormap (like 'RdGy' in the examples). Here's how it helps:

1.Interpretation of Colors: When you use a colormap, different colors represent different Z values (the height or intensity of the function being plotted). Without a color bar, it's impossible to know what specific Z value each color corresponds to. The colorbar() provides a visual scale that explicitly maps the colors used in the contour lines or filled regions to their numerical Z values.

2.Quantitative Understanding: It allows for a quantitative understanding of the plot. You can see at a glance the range of Z values and how they are distributed across the plot's surface. For example, you can quickly identify areas of high or low values and their approximate magnitudes.

3.Clarity and Readability: It significantly enhances the clarity and readability of the plot, making it easier for anyone viewing it to understand the data being presented without needing external information.

Histogram, Binning Plots

A histogram is a graphical representation of a grouped frequency distribution with continuous classes. It is an area diagram and can be defined as a set of rectangles with bases along with the intervals between class boundaries and with areas proportional to frequencies in the corresponding classes. In such representations, all the rectangles are adjacent since the base covers the intervals between class boundaries. The heights of rectangles are proportional to corresponding frequencies of similar classes and for different classes, the heights will be proportional to corresponding frequency densities.

In other words, a histogram is a diagram involving rectangles whose area is proportional to the frequency of a variable and width is equal to the class interval.

The histogram graph is used under certain conditions. They are:

1.The data should be numerical.

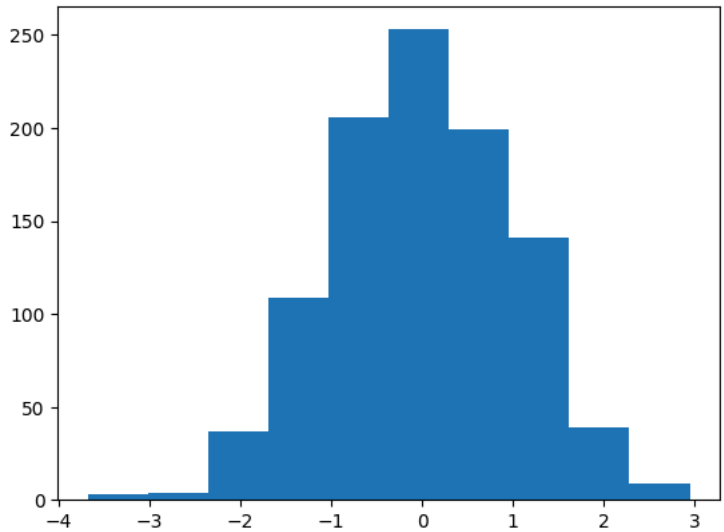
2.A histogram is used to check the shape of the data distribution.

3.Used to check whether the process changes from one period to another.

4.Used to determine whether the output is different when it involves two or more processes.

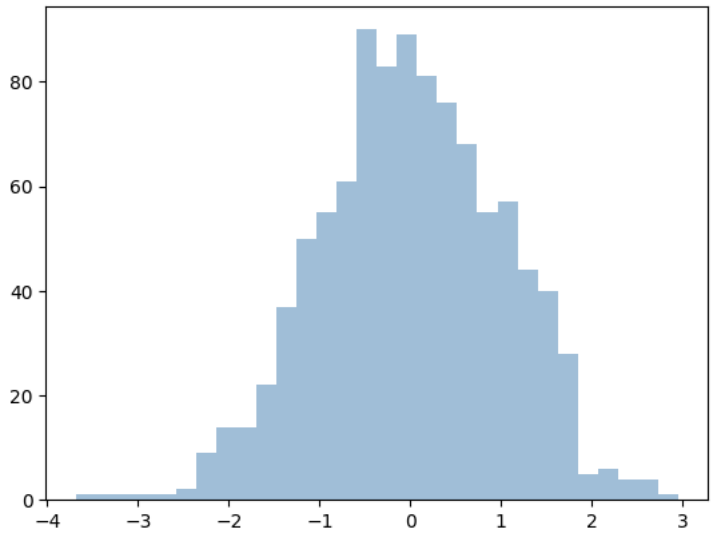
5.Used to analyse whether the given process meets the customer requirements

```
import numpy as np
import matplotlib.pyplot as plt

data = np.random.randn(1000)
plt.hist(data)
plt.show()
```



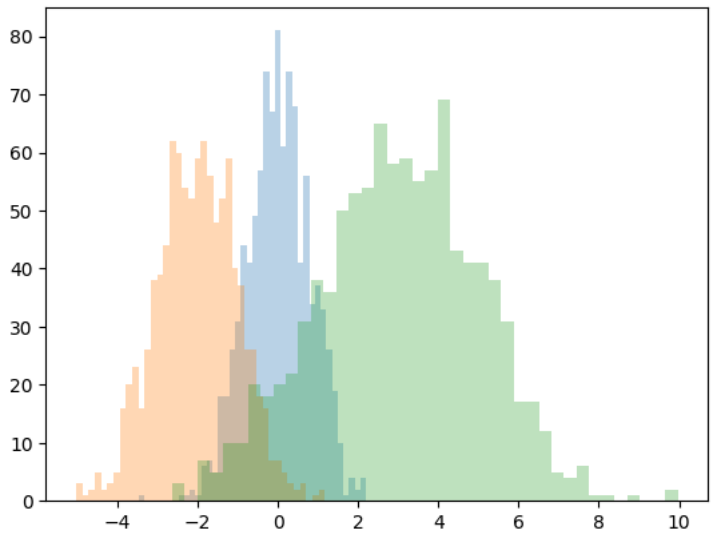
The hist() function has many options to tune both the calculation and the display; here's an example of a more customized histogram:

```
plt.hist(data, bins=30, alpha=0.5, histtype='stepfilled', color='steelblue', edgecolor='none');
```



The plt.hist docstring has more information on other customization options available. I find this combination of histtype='stepfilled' along with some transparency alpha to be very useful when comparing histograms of several distributions:

```
x1 = np.random.normal(0, 0.8, 1000)
x2 = np.random.normal(-2, 1, 1000)
x3 = np.random.normal(3, 2, 1000)

kwargs = dict(histtype='stepfilled', alpha=0.3, bins=40)

plt.hist(x1, **kwargs)
plt.hist(x2, **kwargs)
plt.hist(x3, **kwargs);
```



If you would like to simply compute the histogram (that is, count the number of points in a given bin) and not display it, the np.histogram() function is available

```
#Only storing histogram bin frequencies without ploting it
counts, bin_edges = np.histogram(data, bins=5)
print(counts,bin_edges)
```

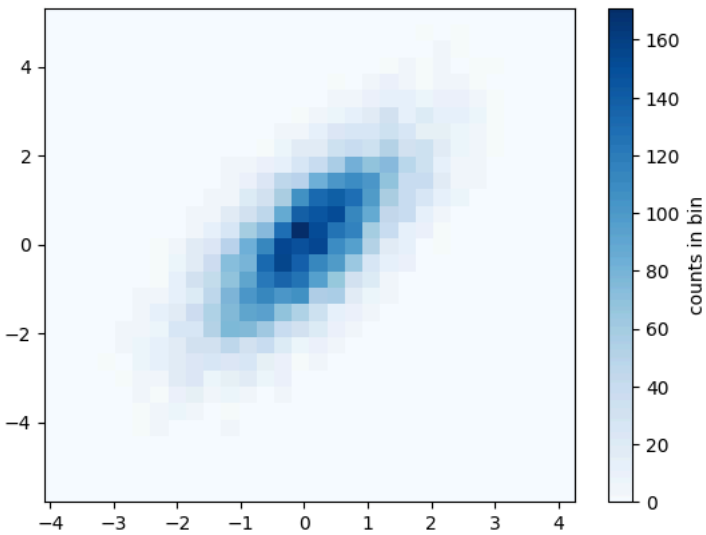
```
[  7 146 459 340  48] [-3.68272679 -2.35662348 -1.03052016  0.29558316  1.62168647  2.94778979]
```

plt.hist2d: Two-dimensional histogram

One straightforward way to plot a two-dimensional histogram is to use Matplotlib's plt.hist2d function:

```
# Create 2 Variable normally distributed data sets with 10000
mean = [0, 0]
cov = [[1, 1], [1, 2]]
x, y = np.random.multivariate_normal(mean, cov, 10000).T
```

```
#Using 2d Histogram plot
plt.hist2d(x, y, bins=30, cmap='Blues')
cb = plt.colorbar()
cb.set_label('counts in bin')
```



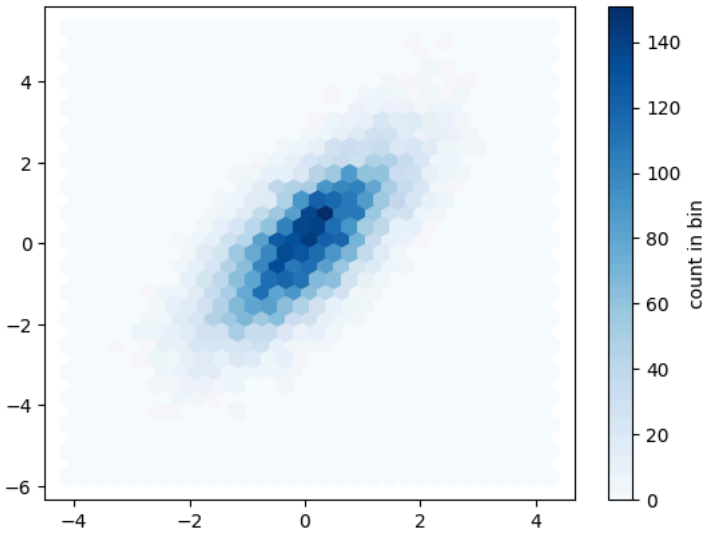
Just as with plt.hist, plt.hist2d has a number of extra options to fine-tune the plot and the binning, which are nicely outlined in the function docstring. Further, just as plt.hist has a counterpart in np.histogram, plt.hist2d has a counterpart in np.histogram2d, which can be used as follows:

```
counts, xedges, yedges = np.histogram2d(x, y, bins=30)
```

plt.hexbin: Hexagonal binnings

The two-dimensional histogram creates a tesselation of squares across the axes. Another natural shape for such a tesselation is the regular hexagon. For this purpose, Matplotlib provides the plt.hexbin routine, which will represents a two-dimensional dataset binned within a grid of hexagons:

```
plt.hexbin(x, y, gridsize=30, cmap='Blues')
cb = plt.colorbar(label='count in bin')
```



Questions:

1.What is role of bins parameter in histogram plot?

The bins parameter in a histogram plot defines the number or the edges of the intervals (bins) into which the data is divided. Each bin then counts how many data points fall within its range.

1.If bins is an integer, it specifies the number of equal-width bins in the range of the data.

2.If bins is a sequence (e.g., a list or array), it defines the explicit edges of the bins. This means you can have bins of unequal width.

Changing the number of bins can significantly alter the appearance of the histogram and how the distribution of the data is perceived.

2.Create a set of 10000 uniform random samples and store it in variable named norm_data. Draw histograms of norm_data with following bin counts 5, 50, 500, 5000.

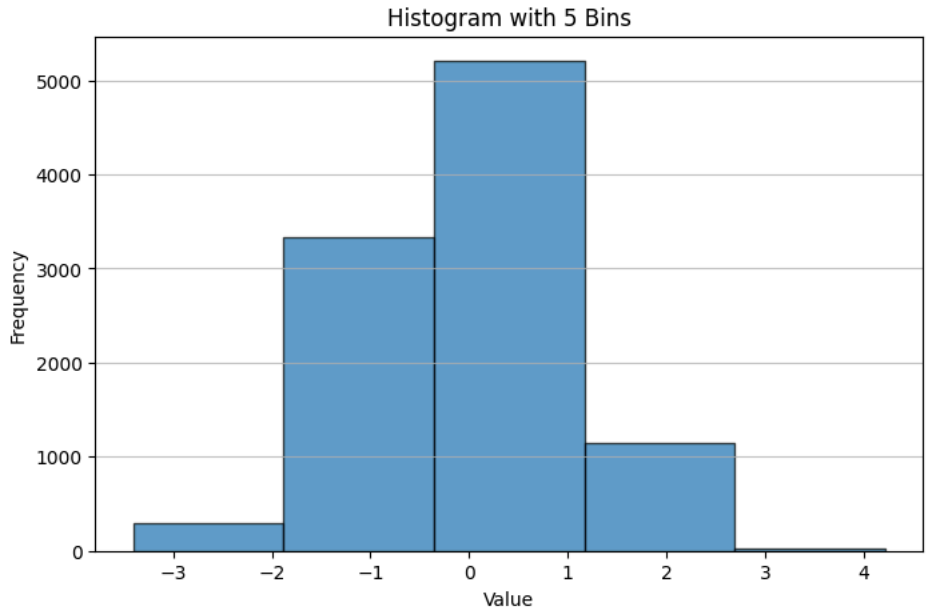
1.Histogram with 5 Bins: This plot provides a very coarse view of the data distribution, showing only broad trends.

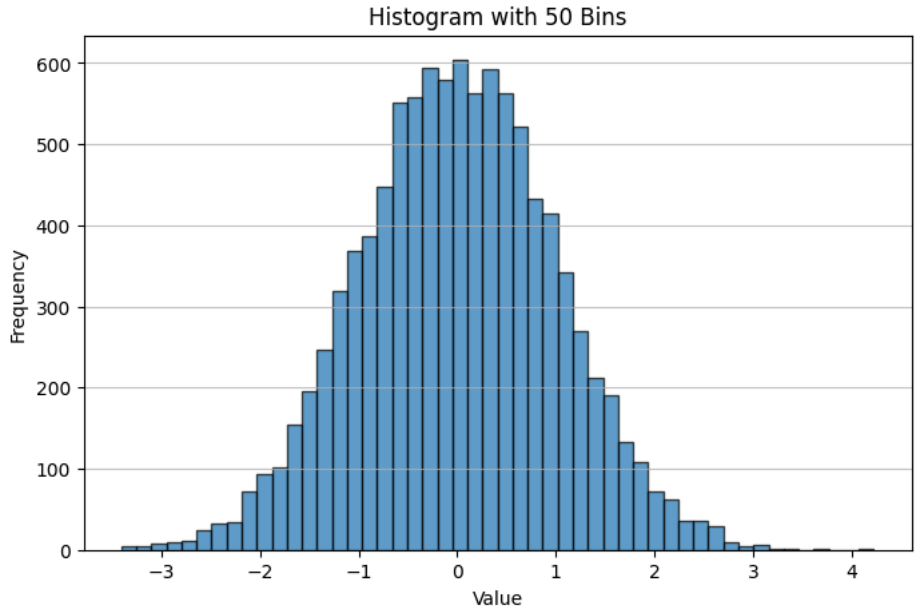
2.Histogram with 50 Bins: This plot offers a much more detailed view, starting to reveal the characteristic bell shape of a normal distribution.

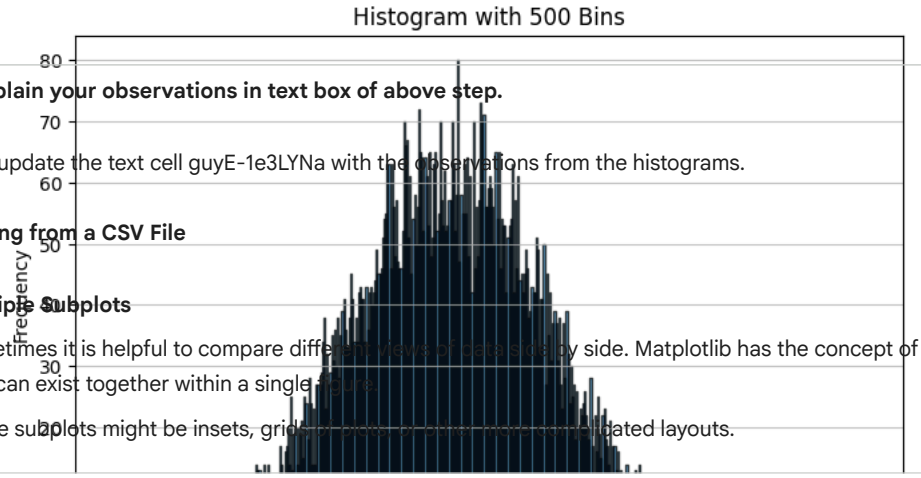
3.Histogram with 500 Bins: This plot shows even finer details of the distribution, making the shape very clear, but might also highlight minor fluctuations due to random sampling.

4.Histogram with 5000 Bins: This plot is extremely detailed, and while it shows the underlying distribution, it also highlights the 'noise' from individual data points, making the bars very thin and potentially jagged, which might make it harder to discern the overall shape compared to the 50 or 500 bin plots

```
import numpy as np
import matplotlib.pyplot as plt

# Create 10000 normally distributed random samples
norm_data = np.random.randn(10000)

# Define the bin counts to use
bin_counts = [5, 50, 500, 5000]

# Draw histograms for each bin count
for bins in bin_counts:
    plt.figure(figsize=(8, 5))
    plt.hist(norm_data, bins=bins, edgecolor='black', alpha=0.7)
    plt.title(f'Histogram with {bins} Bins')
    plt.xlabel('Value')
    plt.ylabel('Frequency')
    plt.grid(axis='y', alpha=0.75)
    plt.show()
```







3.Explain your observations in text box of above step.

I will update the text cell guyE-1e3LYNa with the observations from the histograms.

Ploting from a CSV File

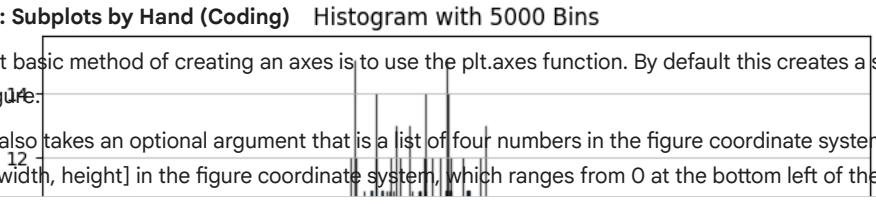
Multiple Subplots

Sometimes it is helpful to compare different views of data side by side. Matplotlib has the concept of subplots : groups of smaller axes that can exist together within a single figure.

These subplots might be insets, grids, or more complicated layouts.

```
import matplotlib.pyplot as plt
import numpy as np

plt.style.use('seaborn-v0_8-white')
```

plt.axes: Subplots by Hand (Coding)



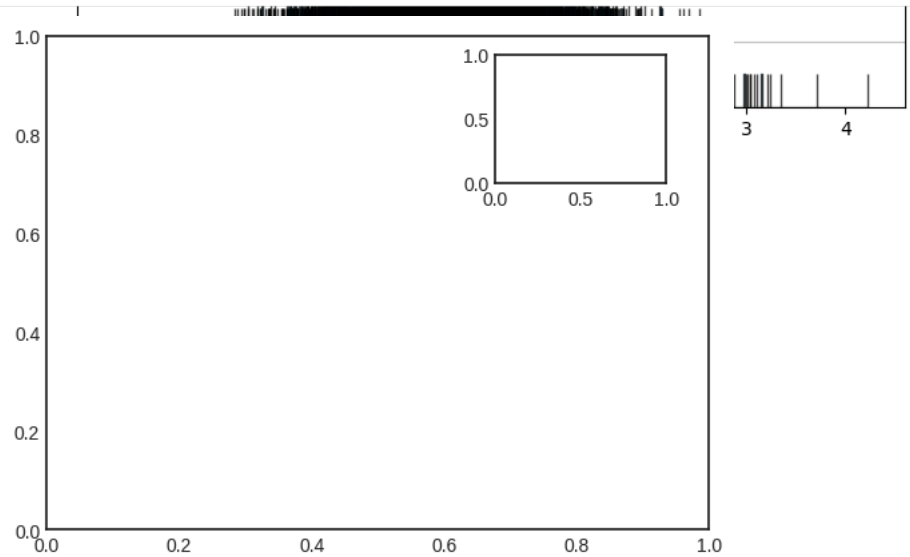
The most basic method of creating an axes is to use the plt.axes function. By default this creates a standard axes object that fills the entire figure.

plt.axes also takes an optional argument that is a list of four numbers in the figure coordinate system. These numbers represent [left, bottom, width, height] in the figure coordinate system, which ranges from 0 at the bottom left of the figure to 1 at the top right of the

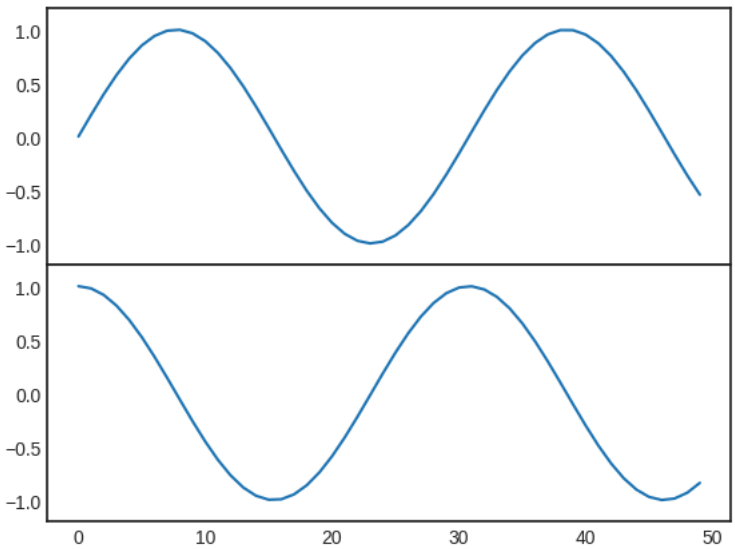
figure.

For example, we might create an inset axes at the upper right corner of another axes by setting the x and y position to 0.65 (that is, starting at 65% of the width and 65% of the height of the figure) and the x and y extents to 0.2 (that is, the size of the axes is 20% of the width and 20% of the height of the figure):

```
ax1 = plt.axes()  # standard axes
ax2 = plt.axes([0.650, 0.650, 0.2, 0.2]) #inset axes
```




```
#creating subplot using fig.add_axes
fig = plt.figure()
ax1 = fig.add_axes([0.1, 0.5, 0.8, 0.4],
                   xticklabels=[], ylim=(-1.2, 1.2))
ax2 = fig.add_axes([0.1, 0.1, 0.8, 0.4],
                   ylim=(-1.2, 1.2))

x = np.linspace(0, 10)
ax1.plot(np.sin(x))
ax2.plot(np.cos(x));
```



plt.subplot: Simple Grids of Subplots

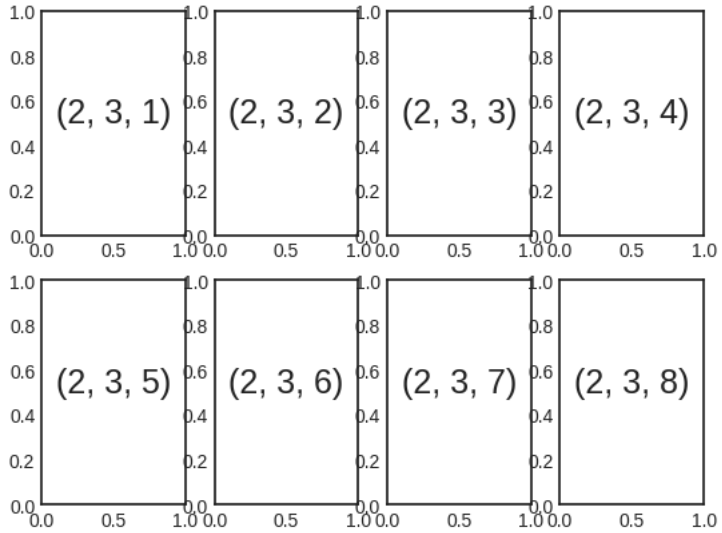
Aligned columns or rows of subplots are a common-enough need that Matplotlib has several convenience routines that make them easy to create.

The lowest level of these is plt.subplot(), which creates a single subplot within a grid.

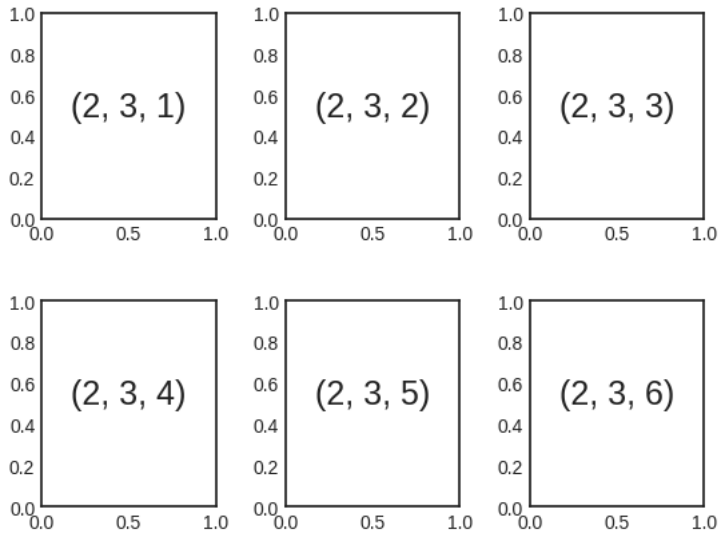
As you can see, this command takes three integer arguments—the number of rows, the number of columns, and the index of the plot to be created in this scheme, which runs from the upper left to the bottom right

```
for i in range(1, 9):
    plt.subplot(2, 4, i)
```

```
    plt.text(0.5, 0.5, str((2, 3, i)),
             fontsize=18, ha='center')
```




```
fig = plt.figure()
fig.subplots_adjust(hspace=0.4, wspace=0.4)
for i in range(1, 7):
    ax = fig.add_subplot(2, 3, i)
    ax.text(0.5, 0.5, str((2, 3, i)),
           fontsize=18, ha='center')
```

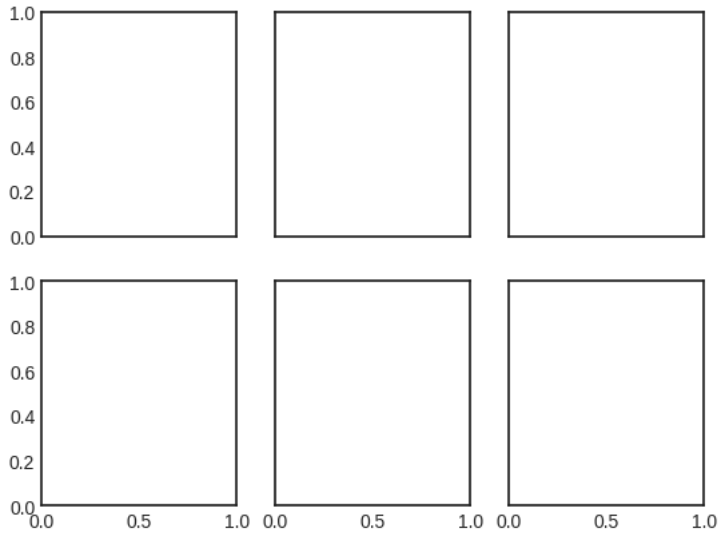


plt.subplots: The Whole Grid in One Go

The approach just described can become quite tedious when creating a large grid of subplots, especially if you'd like to hide the x- and y-axis labels on the inner plots. For this purpose, plt.subplots() is the easier tool to use (note the s at the end of subplots). Rather than creating a single subplot, this function creates a full grid of subplots in a single line, returning them in a NumPy array. The arguments are the number of rows and number of columns, along with optional keywords sharex and sharey, which allow you to specify the relationships between different axes.

Here we'll create a grid of subplots, where all axes in the same row share their y-axis scale, and all axes in the same column share their x-axis scale:

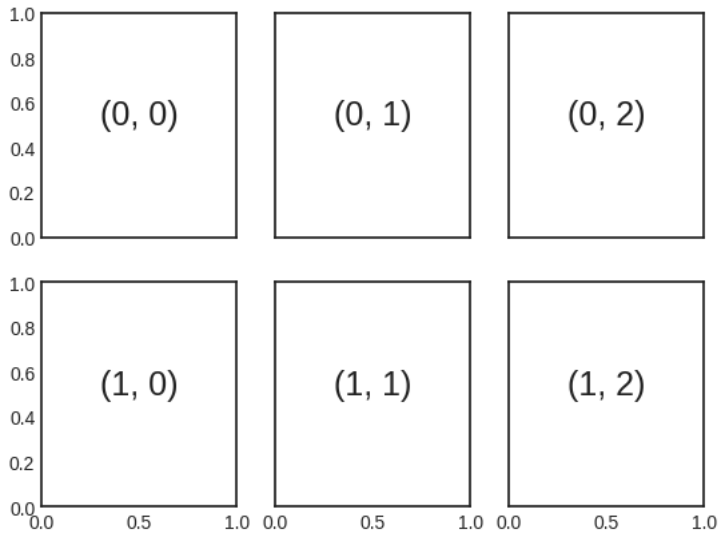
```
fig, ax = plt.subplots(2, 3, sharex='col', sharey='row')
```



Note that by specifying sharex and sharey, we've automatically removed inner labels on the grid to make the plot cleaner.

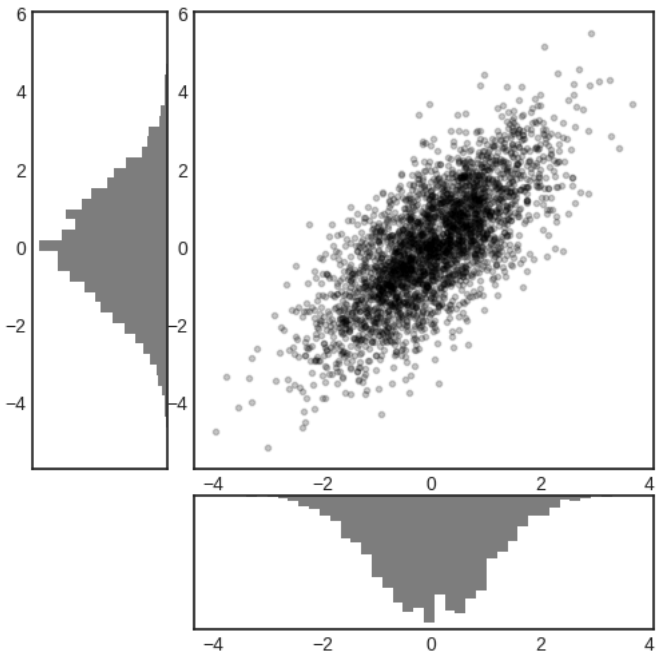
The resulting grid of axes instances is returned within a NumPy array, allowing for convenient specification of the desired axes using standard array indexing notation:

```
# axes are in a two-dimensional array, indexed by [row, col]
for i in range(2):
    for j in range(3):
        ax[i, j].text(0.5, 0.5, str((i, j)),
                      fontsize=18, ha='center')
fig
```




```
# An example of Flexible grid arrangement
# Create some normally distributed data
mean = [0, 0]
cov = [[1, 1], [1, 2]]
x, y = np.random.multivariate_normal(mean, cov, 3000).T

# Set up the axes with gridspec
fig = plt.figure(figsize=(6, 6))
grid = plt.GridSpec(4, 4, hspace=0.2, wspace=0.2)
main_ax = fig.add_subplot(grid[:-1, 1:])
y_hist = fig.add_subplot(grid[:-1, 0], xticklabels=[], sharey=main_ax)
x_hist = fig.add_subplot(grid[-1, 1:], yticklabels=[], sharex=main_ax)

# scatter points on the main axes
main_ax.plot(x, y, 'ok', markersize=3, alpha=0.2)

# histogram on the attached axes
x_hist.hist(x, 40, histtype='stepfilled',
            orientation='vertical', color='gray')
x_hist.invert_yaxis()
```

```
y_hist.hist(y, 40, histtype='stepfilled',
            orientation='horizontal', color='gray')
y_hist.invert_xaxis()
```

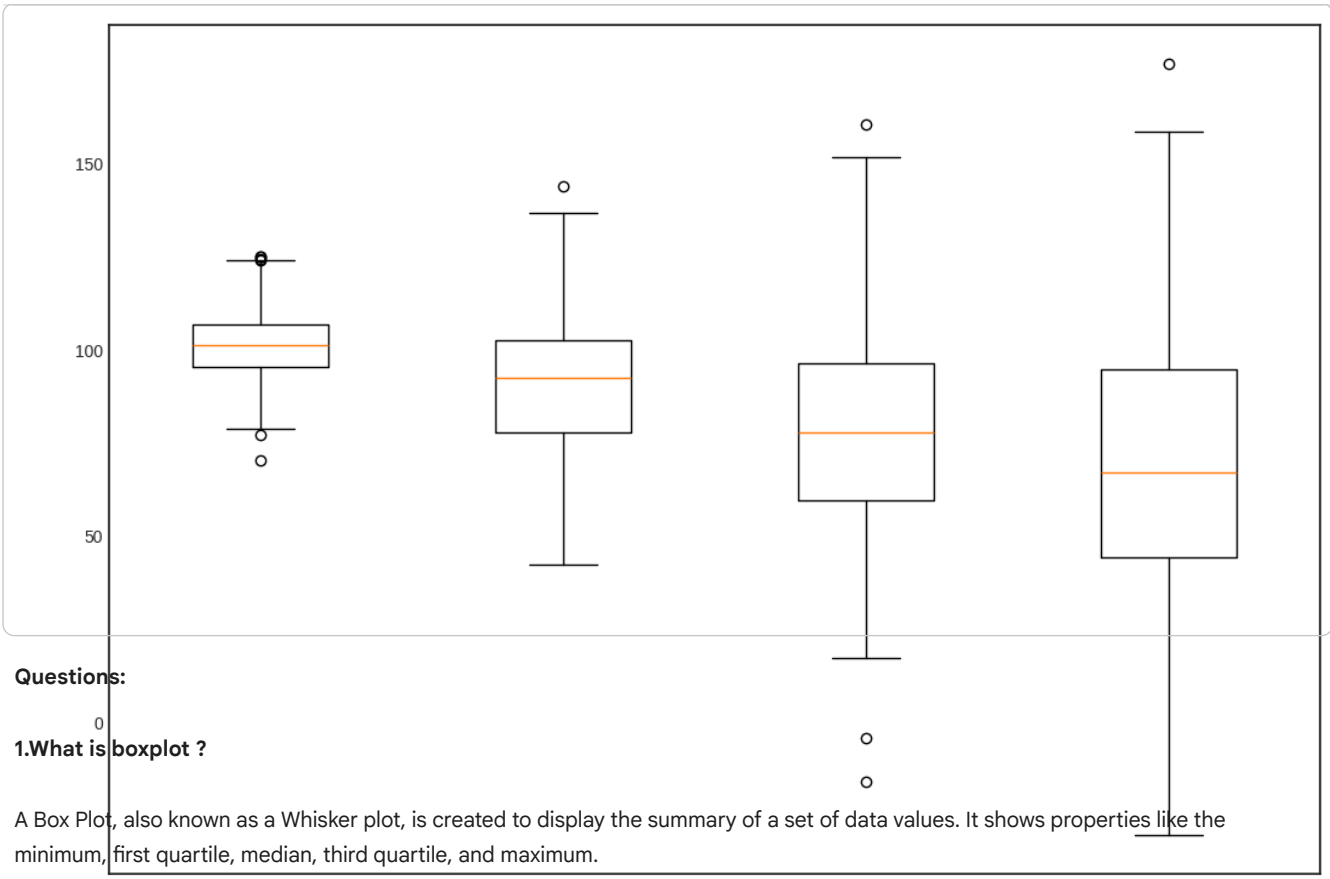


Boxplot

A Box Plot is also known as Whisker plot is created to display the summary of the set of data values having properties like minimum, first quartile, median, third quartile and maximum.

In the box plot, a box is created from the first quartile to the third quartile, a vertical line is also there which goes through the box at the median. Here x-axis denotes the data to be plotted while the y-axis shows the frequency distribution.

```
#Exmple of Box plot for some random data sets
# Import libraries
import matplotlib.pyplot as plt
import numpy as np


# Creating dataset
np.random.seed(10)

data_1 = np.random.normal(100, 10, 200)
data_2 = np.random.normal(90, 20, 200)
data_3 = np.random.normal(80, 30, 200)
data_4 = np.random.normal(70, 40, 200)
data = [data_1, data_2, data_3, data_4]

fig = plt.figure(figsize =(10, 7))

# Creating axes instance
ax = fig.add_axes([0, 0, 1, 1])

# Creating plot
bp = ax.boxplot(data)

# show plot
plt.show()
```



Questions:

1.What is boxplot ?

A Box Plot, also known as a Whisker plot, is created to display the summary of a set of data values. It shows properties like the minimum, first quartile, median, third quartile, and maximum.

In a box plot, a box is drawn from the first quartile to the third quartile, and a vertical line goes through the box at the median. The x-axis typically denotes the data to be plotted, while the y-axis shows the frequency distribution.

2.What information do we get from a box plot?

A box plot provides a wealth of information about the distribution of a dataset. From a box plot, you can clearly see:

1.Median (Q2): The line inside the box represents the median, which is the middle value of the dataset, dividing it into two equal halves.

2.First Quartile (Q1): The bottom of the box marks the first quartile, meaning 25% of the data falls below this value.

3.Third Quartile (Q3): The top of the box marks the third quartile, meaning 75% of the data falls below this value (or 25% falls above it).

4.Interquartile Range (IQR): The length of the box itself (Q3 - Q1) represents the IQR, which is a measure of the spread of the middle 50% of the data.

5.Whiskers: These lines extend from the box to the minimum and maximum values within a certain range (typically 1.5 times the IQR from the quartiles). They show the spread of the bulk of the data, excluding outliers.

6.Outliers: Individual points plotted beyond the whiskers are considered outliers, indicating values that are significantly different from the rest of the dataset.

7.Symmetry/Skewness: The position of the median within the box and the relative lengths of the whiskers can give an indication of the data's symmetry or skewness.

8.Data Spread: The overall length from one end of the whiskers to the other gives a sense of the total spread of the data, excluding extreme outliers.

3.How is minimum and maximum whisker's marked in a box plot ?

The minimum and maximum whiskers in a box plot are typically determined based on the Interquartile Range (IQR) and are drawn to indicate the spread of the data within a certain range, excluding outliers. Here's how they are generally marked:

1.Upper Whisker: This whisker extends from the top of the box (Q3, the third quartile) up to the highest data point that is within 1.5 times the IQR above Q3. Any data points beyond this upper whisker are considered outliers.